



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«МИРЭА – Российский технологический университет»
РТУ МИРЭА**

Институт Информационных Технологий

Кафедра Вычислительной техники

**ОТЧЕТ О ВЫПОЛНЕНИИ ПРАКТИЧЕСКОЙ РАБОТЫ
№4**

«Реализация лексического анализатора на Python»

по дисциплине

«Теория формальных языков»

Выполнил студент группы ИКБО-43-23

Кощев М. И.

Принял ассистент

Цынгальев П.С.

Практическая работа
выполнена

«12» декабря 2024 г.

«Зачтено»

«__» _____ 2024 г.

Москва 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ЦЕЛЬ	4
2 ЗАДАНИЕ.....	5
2.1 Формулировка задания.....	5
2.2 Математическая модель.....	5
2.3 Тестирование программы.....	7
3 ЗАКЛЮЧЕНИЕ.....	12
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....	13
ПРИЛОЖЕНИЯ.....	14

ВВЕДЕНИЕ

Изучение теории формальных языков представляет собой значимый элемент образовательной программы по направлению программной инженерии. Эта область, являющаяся ключевой частью компьютерных наук и теоретической информатики, сосредотачивается на исследовании структуры и свойств формальных языков, а также на моделировании их обработки. В условиях современного общества, где возрастают объемы данных и потребность в эффективных методах их хранения, передачи и обработки, понимание основ формальных языков приобретает особую значимость.

Формальные языки находят применение в различных сферах: от описания синтаксиса языков программирования до разработки сетевых протоколов и создания компиляторов. С их помощью удастся формализовать взаимодействие как между человеком и компьютером, так и между компонентами программного обеспечения. В центре изучения теории формальных языков находятся такие понятия, как грамматики, автоматы и алгебраические структуры, которые позволяют описывать синтаксис и семантику языков.

В данном отчете мы рассмотрим программу реализацию лексического анализатора на языке Python.

1 ЦЕЛЬ

Реализовать лексический анализатор на языке Python, согласно варианту КР.

2 ЗАДАНИЕ

2.1 Формулировка задания

Согласно индивидуальному варианту задания на курсовую работу грамматика языка включает следующие синтаксические конструкции:

Листинг 2.1.1

```
<операции_группы_отношения> ::= <> | = | < | <= | > | >=
<операции_группы_сложения> ::= + | - | or
<операции_группы_умножения> ::= * | / | and
<унарная_операция> ::= not
<программа> ::= program var <описание> begin <оператор> {;
<оператор>} end.
<описание> ::= {<идентификатор> {, <идентификатор> } : <тип>
; }
<тип> ::= % | ! | $
<оператор> ::= <составной> | <присваивания> | <условный> |
<фиксированного_цикла> | <условного_цикла> | <ввода> | <вывода>
<составной> ::= «[» <оператор> { ( : | перевод строки)
<оператор> } «]»
<присваивания> ::= <идентификатор> as <выражение>
<условный> ::= if <выражение> then <оператор> [ else
<оператор>]
<фиксированного_цикла> ::= for <присваивания> to <выражение>
do <оператор>
<условного_цикла> ::= while <выражение> do <оператор>
<ввода> ::= read «(»<идентификатор> {, <идентификатор> } «)»
<вывода> ::= write «(»<выражение> {, <выражение> } «)»
{ ... }
<выражение> ::= <операнд> {<операции_группы_отношения>
<операнд>}
<операнд> ::= <слагаемое> {<операции_группы_сложения>
<слагаемое>}
```

```
<слагаемое> ::= <множитель> {<операции_группы_умножения>
<множитель>}
<множитель> ::= <идентификатор> | <число> | <логическая_константа> |
<унарная_операция> <множитель> | «(><выражение><)>»
<число> ::= <целое> | <действительное>
<логическая_константа> ::= true | false
<идентификатор> ::= <буква> {<буква> | <цифра>}
<буква> ::= A | B | C | D | E | F | G | H | I | J | K | L | M
| N | O | P | Q | R | S | T | U | V | W | X | Y | Z | a | b | c | d
| e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t |
u | v | w | x | y | z
<цифра> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

2.2 Математическая модель

Лексический анализатор – подпрограмма, которая принимает на вход исходный текст программы и выдает последовательность лексем – минимальных элементов программы, несущих смысловую нагрузку.

В модельном языке программирования выделяют следующие типы лексем:

- ключевые слова;
- ограничители;
- числа;
- идентификаторы.

При разработке лексического анализатора, ключевые слова и ограничители известны заранее, идентификаторы и числовые константы –вычисляются в момент разбора исходного текста. Для каждого типа лексем предусмотрена отдельная таблица. Таким образом, внутреннее представление лексемы – пара чисел (n, k), где n – номер таблицы лексем, k – номер лексемы в таблице.

Кроме того, в исходном коде программы кроме ключевых слов, идентификаторов и числовых констант может находиться произвольное число

пробельных символов («пробел», «табуляция», «перенос строки», «возврат каретки») и комментариев, заключенных в фигурные скобки.

Исходные код лексического анализатора приведен в Приложении А.

Рисунок 1 – Диаграмма состояний лексического анализатора

Разработано консольное приложение, предназначенное для анализа исходного текста программы, написанной на модельном языке. Программа делает лексический анализ введенного кода. В результате выполнения приложение либо подтверждает правильность программы, либо выводит сообщение об ошибке. Рассмотрим несколько примеров работы приложения. Рассмотрим несколько примеров работы приложения. Исходный код программы приведен в Листинге 2.3.1.

Листинг 2.3.1 – main.py

```
program
var x, y : integer;
begin
    x := 5; { Присваиваем x значение 5 }
    y := 10; { Присваиваем y значение 10 }
write (x);
write (y);
end.
```



```
* Лексический анализ *
Токен: KEYWORD, элемент: program
Токен: KEYWORD, элемент: var
Токен: ID, элемент: x
Токен: DELIMITER, элемент: ,
Токен: ID, элемент: y
Токен: DELIMITER, элемент: :
Токен: KEYWORD, элемент: integer
Токен: DELIMITER, элемент: ;
Токен: KEYWORD, элемент: begin
Токен: ID, элемент: x
Токен: ASSIGN, элемент: :=
Токен: NUMBER, элемент: 5
Токен: DELIMITER, элемент: ;
Токен: ID, элемент: y
Токен: ASSIGN, элемент: :=
Токен: NUMBER, элемент: 10
Токен: DELIMITER, элемент: ;
Токен: KEYWORD, элемент: write
Токен: DELIMITER, элемент: (
Токен: ID, элемент: x
Токен: DELIMITER, элемент: )
Токен: DELIMITER, элемент: ;
Токен: KEYWORD, элемент: write
Токен: DELIMITER, элемент: (
Токен: ID, элемент: y
Токен: DELIMITER, элемент: )
Токен: DELIMITER, элемент: ;
Токен: KEYWORD, элемент: end
Токен: DELIMITER, элемент: .
* Лексический анализ завершен *
```

Рисунок 2.3.1 – Тестирование лексического анализа

Исходный код программы, содержащий лексическую ошибку, приведен в листинге 2.3.2.

Листинг 2.3.2 – main.py

```
program
var x, y : integer;
    x := 5; { Присваиваем x значение 5 }
    y := 10; { Присваиваем y значение 10 }
write (x);
write (y);
end.
```

```
* Лексический анализ *
Токен: KEYWORD, элемент: program
Токен: KEYWORD, элемент: var
Токен: ID, элемент: x
Токен: DELIMITER, элемент: ,
Токен: ID, элемент: y
Токен: DELIMITER, элемент: :
Токен: KEYWORD, элемент: integer
Токен: DELIMITER, элемент: ;
Токен: ID, элемент: x
Токен: ASSIGN, элемент: :=
Токен: NUMBER, элемент: 5
Токен: DELIMITER, элемент: ;
Токен: ID, элемент: y
Токен: ASSIGN, элемент: :=
Токен: NUMBER, элемент: 10
Токен: DELIMITER, элемент: ;
Токен: KEYWORD, элемент: write
Токен: DELIMITER, элемент: (
Токен: ID, элемент: x
Токен: DELIMITER, элемент: )
Токен: DELIMITER, элемент: ;
Токен: KEYWORD, элемент: write
Токен: DELIMITER, элемент: (
Токен: ID, элемент: y
Токен: DELIMITER, элемент: )
Токен: DELIMITER, элемент: ;
Токен: KEYWORD, элемент: end
Токен: DELIMITER, элемент: .
* Лексический анализ завершен с ошибкой*
```

Рисунок 2.3.1 – Тестирование ошибки лексического анализатора

3 ЗАКЛЮЧЕНИЕ

В процессе выполнения практической работы были достигнуты следующие результаты:

Разработан лексический анализатор, позволяющий разделить последовательность символов исходного текста программы на последовательность лексем. Лексический анализатор реализован на языке Python в виде класса `LexicalAnalyzer`.

Реализован лексический анализатор индивидуального варианта КР на ЯП Python. Проведено тестирование программы, и результаты представлены в отчете. Таким образом, поставленная цель работы — Реализовать лексический анализатор на языке Python, согласно варианту КР — успешно достигнута.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Малявко А.А. Формальные языки и компиляторы: учебное пособие для вузов. - М., Изд-во Юрайт, 2019.
2. Свердлов С.З. Языки программирования и методы трансляции : учебное пособие для вузов - СПб., 2007, Id=65534.
3. Карпов Ю.Г. Теория и технология программирования. Основы построения трансляторов: учеб. пособие. - СПб.: БХВ-Петербург, 2005, Id=64347.

ПРИЛОЖЕНИЯ

Приложение А – код на языке Python

ПРИЛОЖЕНИЕ А

Код на языке Python

Листинг А.1 – *lexer.py*

```
class LexicalAnalyzer:
    class State(Enum):
        ID = "ID" # Идентификаторы
        NUM = "NUM" # Числа
        COM = "COM" # Комментарии
        ALE = "ALE" # Операции отношения
        NEQ = "NEQ" # Неравенство
        DELIM = "DELIM" # Разделители
        STR = "STR" # Строковые литералы

    # Ключевые слова
    TW = [
        "program", "var", "begin", "end", "if", "else",
        "while", "for", "to", "then", "next", "as",
        "readln", "write", "true", "false", "%", "!", "$",
        "end_else", "real", "integer", "boolean"
    ]

    # Разделители и операторы
    TD = [
        "[", "]", "{", "}", "(", ")", ",", ":", ";", ":", ":",
        ".", "+", "-", "*", "/", "and", "/", "not",
        "!", "=", "<", "<=", ">", ">="
    ]

    def __init__(self, input_text):
        self.text = input_text
        self.pos = 0
        self.current_char = self.text[self.pos] if
self.text else None
        self.tokens = []
        self.before_begin = True

    def advance(self):
        self.pos += 1
        self.current_char = self.text[self.pos] if self.pos
< len(self.text) else None

    def add_token(self, type_, value):
        self.tokens.append((type_, value))
```

```
def clear_whitespace(self):
    while self.current_char and self.current_char in '
\n\r\t':
        self.advance()

def parse_identifier_or_keyword(self):
    start = self.pos
    while self.current_char and
(self.current_char.isalnum() or self.current_char == '_'):
        self.advance()
    text = self.text[start:self.pos]
    if text in self.TW:
        self.add_token('KEYWORD', text, )
        if text == 'begin':
            self.before_begin = False
    else:
        self.add_token('ID', text, )

def parse_number(self):
    start = self.pos
    base_detected = False
    if self.current_char == '0':
        self.advance()
        if self.current_char in 'Bb':
            base_detected = True
            self.advance()
            while self.current_char and
self.current_char in '01':
                self.advance()
            elif self.current_char in 'Oo':
                base_detected = True
                self.advance()
                while self.current_char and
self.current_char in '01234567':
                    self.advance()
                elif self.current_char in 'Xx':
                    base_detected = True
                    self.advance()
                    while self.current_char and
(self.current_char.isdigit() or self.current_char.upper() in
'ABCDEF'):
                        self.advance()
                if not base_detected:
                    while self.current_char and
self.current_char.isdigit():
                        self.advance()
```



```

        if self.current_char == '.':
            self.advance()
            while self.current_char and
self.current_char.isdigit():
                self.advance()
            if self.current_char and
self.current_char.upper() == 'E':
                self.advance()
                if self.current_char in '+-':
                    self.advance()
                if self.current_char and
self.current_char.isdigit():
                    while self.current_char and
self.current_char.isdigit():
                        self.advance()
                    else:
                        self.add_token('ERROR',
self.text[start:self.pos], )
                        return
                if self.current_char in 'bohBOH':
                    suffix = self.current_char.lower()
                    self.advance()
                    self.add_token('NUMBER',
self.text[start:self.pos] + suffix, )
                else:
                    text = self.text[start:self.pos]
                    self.add_token('NUMBER', text, )

    def parse_string(self):
        self.advance()
        start = self.pos
        while self.current_char and self.current_char !=
"":
            self.advance()
            text = self.text[start:self.pos]
            self.add_token('STRING', f"'{text}'", )
            self.advance()

    def parse_comment(self):
        self.advance()
        while self.current_char and self.current_char !=
'}':
            self.advance()
            self.advance()

    def parse_delimiter_or_operator(self):
        start = self.pos
        self.advance()
        if self.current_char and (self.text[start:self.pos
+ 1] in self.TD):
            self.advance()

```

```

text = self.text[start:self.pos]
    if text in ["==", "!=", "<", "<=", ">", ">="]:
        self.add_token('REL_OP', text, )
    elif text in ["+", "-", "||"]:
        self.add_token('ADD_OP', text, )
    elif text in ["*", "/", "&&"]:
        self.add_token('MUL_OP', text, )
    elif text == "!=":
        self.add_token('ASSIGN', text, )
    elif text in self.TD:
        self.add_token('DELIMITER', text, )
    else:
        self.add_token('UNKNOWN', text, )

def tokenize(self):
    while self.current_char:
        self.clear_whitespace()
        if not self.current_char:
            break
        if self.current_char.isalpha():
            self.parse_identifier_or_keyword()
        elif self.current_char.isdigit():
            self.parse_number()
        elif self.current_char == "'":
            self.parse_string()
        elif self.current_char == '{':
            self.parse_comment()
        elif self.current_char == '!':
            if self.text[self.pos:self.pos + 2] ==
"!=":
                self.add_token('REL_OP', '!=', )
                self.advance()
                self.advance()
            else:
                if self.before_begin:
                    self.add_token('KEYWORD', '!', )
                else:
                    self.add_token('DELIMITER', '!', )
                    self.advance()
        elif self.current_char in "%$":
            self.add_token('KEYWORD',
self.current_char, )
            self.advance()
        elif self.current_char in self.TD:
            self.parse_delimiter_or_operator()
        else:
            self.add_token('UNKNOWN',
self.current_char, )
            self.advance()
    return self.tokens

```